2018 42nd IEEE International Conference on Computer Software & Applications

Using Service Dependency Graph to Analyze and Test Microservices

Shang-Pin Ma[1], Chen-Yuan Fan[1], Yen Chuang[1], Wen-Tin Lee[2], Shin-Jie Lee[3], Nien-Lin Hsueh[4]
Department of Computer Science and Engineering, National Taiwan Ocean University, Keelung, Taiwan[1]
Department of Software Engineering and Management, National Kaohsiung Normal University, Kaohsiung, Taiwan[2]
Department of Computer Science and Information Engineering, National Cheng Kung University, Tainan, Taiwan[3]
Department of Computer Science and Information Engineering, Feng Chia University, Taichung, Taiwan[4]
albert@ntou.edu.tw, b4456609@gmail.com, 10557030@ntou.edu.tw, wtlee@nknu.edu.tw, jielee@mail.ncku.edu.tw, nlhsueh@mail.fcu.edu.tw

Abstract—Microservice architecture (MSA) is an emerging software architectural style, which differs fundamentally from the monolithic, layered architecture. MSA is based on microservices to provide several advantages, such as autonomy, composability, scalability, and fault-tolerance. However, how to manage complex "call" relationships between microservices is still a big issue that needs to be addressed. In this paper, we propose an approach for assisting the development of MSA-based systems, referred to as GMAT (Graph-based Microservice Analysis and Testing). GMAT can automatically generate "Service Dependency Graph (SDG)" to analyze and visualize the dependency relationships between microservices. Using GMAT, people are able to detect anomalies by analyzing risky service invocation chains in early stage of development, and trace the linkages between services when developing a new version of a target system. Experiments show that GMAT is able to work well for both small-scale and large-scale MSA-based systems.

Keywords- microservice testing, microservice analysis, service dependency graph, service engineering

I. Introduction

The rapid development of the internet has greatly increased the number of requests for services on demand. Traditional software applications, referred to as monolithic ones, are divided into the front-end UI (user interface), service-side logic components, and database. Monolithic architecture needs to struggle with handling cases in which the number of users exceeds the capacity of the server. Furthermore, monolithic architectures are hard to manage and maintain due to a lack of mechanisms aimed at modularization [1]. Microservice architecture (MSA) [2-6] is an emerging cloud-based software architecture, which differ fundamentally from monolithic systems. MSA provides intermediate building blocks, referred to as microservices, to handle specific functions. Microservices provide a lot of advantages, such as autonomy, composability, scalability, and fault-tolerance. Several enterprises, such as Netflix, Amazon, and Spotify, have adopted the microservice architecture to build scalable, easily maintainable software systems. However, microservices has two characteristics, Microservice choreography and evolutionary design, that cause several pending issues. The first issue is how to manage complex call relationships between microservices while the second one is how to efficiently assure the quality of continuously changing microservices.

To address the above issues, in this paper, we propose an approach for assisting the development of MSA-based systems, referred to as GMAT (Graph-based Microservice Analysis and Testing). GMAT can automatically generate "Service Dependency Graph (SDG)" to analyze and visualize the dependency relationships between microservices. Using GMAT, people are able to detect anomalies by analyzing risky service invocation chains in early stage of development, and trace the linkages between services when developing a new version of a target system. The system overview of GMAT is shown in Figure 1.

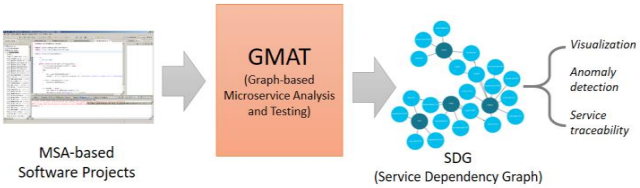

Figure 1 Overview of GMAT approach

The remainder of this paper is organized as follows: Section 2 outlines background knowledge and related work. In Section 3, we describe the proposed GMAT approach in depth. Section 4 presents the quantitative experiments. Section 5 draws a number of conclusions.

II. Background and Related Work

A. Characteristcis of Microservices

An in-depth survey of the literature led to the identification of five characteristics of microservices [2-6]:

1. Autonomous: microservices are responsible for the execution of their own underlying processes, which can be initiated, operated, and shut down independently. They can be also developed and deployed separately within their own development lifecycle.
2. Scalability: microservices are small, stateless components, which makes them easy to scale up.
3. Technology heterogeneity: microservices use unified platform-independent, communication protocols for the exchange of information, which means that they can be constructed using a variety of programming languages on various platforms using many types of database.
4. Specialized: each service handles specific domain logic and communicates with other services using well-designed interfaces.
5. Fault Tolerance: the fact that each microservice has its own lifecycle means that it is able to tolerate failures by applying a failover mechanism or skipping the invocation, whenever a service fails.

0730-3157/18/$31.00 ©2018 IEEE
DOI 10.1109/COMPSAC.2018.10207

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.

For the sake of scalability, an MSA should be equipped with Service Discovery, API Gateway, and Circuit Breaker. Service discovery provides a mechanism for the registration of microservices. This makes the services addressable, such that they are able to communicate with each other using an IP and port [7]. API Gateway is an interface between the client and microservice system, which allows users to invoke the services they require without the need for any knowledge pertaining to the structure of the system. Circuit Breaker is an error handling mechanism to increase the resiliency of the system.

Various automation tools are required for the application of MSA. A pipeline [8] breaks down the software delivery process into stages, including build automation, continuous integration [9], test automation, and deployment automation. The use of containers [10] in the pipeline facilitates deployment and portability to accelerate the delivery of software systems to users. A container provides an environment in which to run an application program, and may be regarded as a lightweight virtual machine not running under Hypervisor. The use of the Docker [11] container is widely recommended to run microservices. In large-scale microservice systems, centralized logs, metrics, and feedback systems can be used to facilitate management and on-going development.

B. Testing Mechanisms for Microservices

Testing is a key step in software development. Within a microservice architecture, the testing process plays an important role in the integration of services in a pipeline. In this phase, we introduce four testing methods including, unit test, service test, end-to-end test, and behavior-driven test. It is generally preferable to minimize the number of test cases to ensure the fastest possible pipeline processing.

To ensure the functionality of a microservice, it must pass unit testing during development. It is common to employ JUnit as a unit testing tool. Mockito is a Mock framework used to verify the behavior of mock objects, such as the time of a function call, and verify the correctness of input parameters and return values.

Service testing must take into account the consumer-driven contract between a service provider and its service consumer to ensure successful service invocation. During consumer-driven contract testing, the unit test is implemented on the consumer side to verify the body and header of the request and define an expectation of the header and body. When a unit test is passed, a DSL (Domain Specific Language) file is auto-generated, The DSL file contains verification information to the API provider. For service testing in the provider side, it is easy to simply use the DSL generated from the service consumer to verify the service provider. Pact is an open-source contract-driven testing tool, which can be used to implement the above-mentioned approach. Pact supports most popular programming languages. Besides, the DSL generated from Pact can be managed by Pact Broker.

The End-to-End Test is used to test whether the system acts as the expected use cases. It is also an important part of the testing of microservice systems. Selenium is an automated tool for testing web pages, which includes a Firefox plugin allowing automated script writing in various programming languages. Video and screenshots can also be captured during testing. Selenium allows the use of Selenium Grid for the concurrent execution of multiple cases to speed up validation. Nightmare is a high-level browser automation tool that can be implemented without the need for an actual browser. It uses Electron, a web technology used in the development of desktop applications, providing simpler syntax and a wider choice of API.

Behavior-driven development (BDD) is a common approach to agile development, allowing analysts, developers, and testers to work together in the development of scenarios. The final step in testing a microservice system is to confirm that the system works well in all anticipated situations. Gherkin is a business-readable, domain-specific language commonly used in behavior-driven testing. It can be used to describe the behavior of software without the need to describe the method of implementation. It also can be used as a document of the software. Cucumber and Serenity BDD are BDD frameworks that support Gherkin as well as a variety of programming language, including Java. It can also be used with JUnit, Selenium, and other test tools. As long as a Gherkin description file is written and tests implemented, Cucumber can be used to perform tests (based on the descriptor) to produce Pretty, HTML, JSON and other test reports.

C. Discovery Service

Within a microservice system, services are able to communicate with each other. However, enabling services to find each other and communicate remains an on-going problem. Introducing a service discovery mechanism makes it possible for a service to obtain real-time information about the IP and Port of other services simply by implementing service discovery. Thus, the service need not be aware of the IP or Port of other services. Besides, since Service Discovery tools enable the mapping from one name to multiple services, it can also provide the functionality of load balancing with simple service selection methods, such as Round Robin or Random selection.

A number of Service Discovery systems are available, including Etcd, Consul, and Eureka. Etcd is based on key/value store. Consul provides built-in auto load balancing in cases where services are registered under the same name. Eureka was developed by Netflix running on JVM with a built-in load balancer. All of the above-mentioned tools have RESTful interface, which is very useful in the development and implementation of microservices.

D. OpenAPI (Swagger)

The OpenAPI Specification (OAS, originally called Swagger) [12] is a document specification for describing, generating, and consuming RESTful Web services. A variety of tools can generate code, documentation and test cases based on OAS. OAS released by Wordnik in 2011, supports lightweight formats like JSON and YAML. Many Web API developers have already supplied their OAS documents to their clients. The content of OAS can be divided into two parts: info and path. The info part includes API title, version, and contact information. The

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.

path part includes multiple objects which are comprising one or more HTTP operations like GET, POST, or DELETE. In operation, it includes not only the description, but also input parameters and responses, which can refer to the external definitions by $ref tag. OAS/Swagger is the most widely-used specification for describing microservices [13].

III. GMAT (Graph-based Microservice Analysis and Testing)

In this section, we introduce the proposed approach, GMAT (Graph-based Microservice Analysis and Testing), in depth. In the beginning, we identified three research goals that GMAT should achieve, as follows.

1. *Visualizing the dependency relationships between microservices*. Extracting the service calls (i.e., service invocations) between microservices, building corresponding service dependency graph, and visualizing the graph is the first and most important research goal of GMAT.
2. *Detecting cyclic dependency references*. The cyclic dependency references between microservices is very risky from the viewpoints of both development and maintenance. Cyclic dependency references may cause failures at runtime and increase the complexity of developing new versions of microservices. Effectively detecting cyclic dependency references is the second research goal.
3. *Improving the coverage of service tests*. To ensure the quality of a developed system, testing microservice invocations is a necessity. A system lacks of complete microservice testing is very fragile and prone to be crash. Meanwhile, cross-service debugging is difficult when a runtime error occurs. Thus, providing an assistant tool to improve the test coverage is the third research goal.

A. Service Dependency Graph (SDG) and Service Invocation Chain (SIC)

A microservice provides APIs to other microservices and also consumes (or calls) other microservices in a system. Service invocations cause dependency relationships between microservices. When the number of microservices is large, it is hard to manage the complex dependency relationships between these microservices. In this research, we propose the concept of service dependency graph to address the above issue. The basic idea is to collect all service invocation links, construct a service dependency graph (SDG) to represent all links, and show the SDG in a visualized way to let users easily browse all service dependency relationships and check anomalies or errors by using the graph. Accordingly, there are three required steps for the construction of visualized SDG: 1) gathering all service invocation links from multiple microservice projects for a MSA-based system; 2) constructing the internal representation of SDG; and 3) displaying the SDG using visualization techniques.

The concept of SDG is very simple, as shown in Figure 2. The dark-blue circle represents a microservice while the light-gray circle represents the service endpoint. A microservice owns one or more endpoints; i.e., a microservice component provides one or more service operations. A directed arrow indicates that a microservice invokes an endpoint of another microservice.

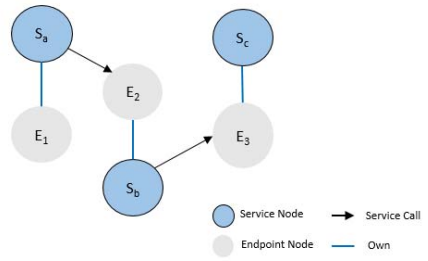

Figure 2 Concept of service dependency graph (SDG)

In this research, we also propose the concept of service invocation chain (SIC) to provide supplementary information for SDG. The service invocation chain indicates a sequence of service invocations between multiple microservices (See the heavy line of Figure 3). GMAT is enabled to find all microservices involved in a SIC and show the SIC in SDG when the user clicks any involved service node. Besides, when a service test case fails, the SIC that involves the failed invocation will be highlighted to effectively help users to identify the error root.

In summary, by using GMAT, SDG could help developers to manage the complex service interactions while SIC could effectively find error roots when failures occur. The main tasks performed by GMAT include: 1) extracting service endpoint information and service call information to construct service dependency relationships; 2) check the status of service tests and automatically generate the test code (using Pact) if some service test cases were not written; 3) combining all information to produce a SDG document and displaying SDG in a visualized way based on the SDG document.

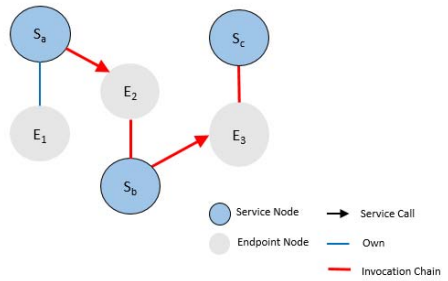

Figure 3 Concept of service invocation chain (SIC)

B. Construction of SDG

How to extract required information for SDG is a key issue for generating SDG. In this research, we apply the *Reflection* mechanism to gather all necessary data, such as service endpoints and service calls. Many programming languages, such as Java, Python, and C#, support the *Reflection* mechanism. Its advantage is that we can obtain various program information without program execution. Using the *Reflection* mechanism, GMAT can retrieve the service endpoints and service call information directly. If the implementation language does not support the feature of reflection, service dependency information could be directly provided by developers via extended Swagger to allow GMAT obtain required data.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.

According to multiple sources [14, 15], Java-based Spring Boot Framework is deemed as the most matured framework for developing microservices. Therefore, we decided to firstly support Spring Boot projects in GMAT. In other words, developers could use the Spring Boot framework to build microservices and follow several simple guidelines and practices to develop microservices so that GMAT can produce visualized SDG automatically. The microservice development guidelines in GMAT include:

- Developers are strongly recommended to use S*pring Boot Actuator* to monitor microservices and gather metrics. GMAT relies on *Spring Boot Actuator* to obtain the endpoint information.
- Developers are also strongly recommended to use *Spring Feign*, a declarative HTTP client developed by Netflix, to invoke other microservices. Spring Feign can work smoothly with *Spring Eureka* to enhance the flexibility by using the discovery service. GMAT relies on *Spring Feign* to retrieve the information of service calls. An example code of Applying *Spring Feign* is shown in Figure 4. The developer has to mark *FeignClient* in the interface and specify the name of the microservice. The specified name needs to be the same as the one registered in *Spring Eureka*. The *FeignClient* also needs to be annotated with *RequestMapping* information to specify the service URL path and service method.

```
@FeignClient (name = "theater")
public interface TheaterClient {
 @RequestMapping (path = "/book", method = RequestMethod.POST)
 BookResultDTO bookShow (@RequestBody BookShowDTO show) ;
}
```

Figure 4 Example of Spring Feign

- Developer are recommended to additionally apply *springfox-swagger2* to automatically generate Swagger based on annotations. GMAT generates reports based on Swagger.

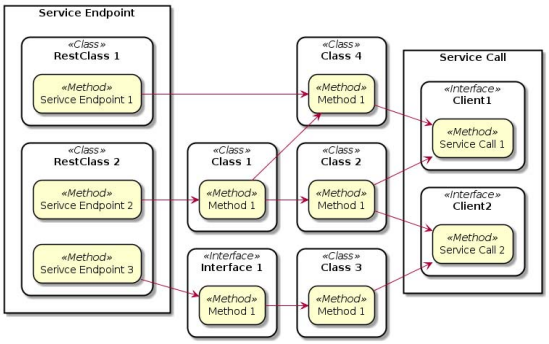

Figure 5 Concept of dependencies between microservices

For the project that follows the above guidelines, GMAT can obtain information of microservice endpoints and service calls via the *Java Reflection* mechanism, and establish relationships between microservices based on these information. When applying the *Reflection*, in order to successfully find the service endpoints and service calls, it needs to deal with the Java interfaces and corresponding implementation classes. Figure 5 is a diagram that shows a microservice project including multiple related interfaces, implementation classes, and their interactions. A bigger rounded rectangle represent a class or an interface. Smaller, colored rounded rectangles inside an interface or a class is a method. In Figure 5, the rectangle in left-hand side is endpoints of the microservice. The rectangle in right-hand side is service call objects. The classes in the middle is core objects encapsulating business logic for the microservice. The arrowed lines represent function calls. In Figure 5, GMAT can extract four links between service endpoints and service calls, including Service Endpoint 1 and Service Call 1, Service Endpoint 2 and Service Call 1, Service Endpoint 2 and Service Call 2, Service Endpoint 3 and Service Call 2. All these links are stored in a graph database, Neo4J[1], for further processing. By inspecting these links of all microservice projects of the target application, GMAT can build all service invocation chains and construct an overall service dependency graph. In other words, GMAT is able to generate visualized SDG based on the collected information of services, service endpoints, and service calls. In this research, we apply D3.js[2], the widely-used web visualization framework, to render the SDG. An example is shown in Figure 6. Developers could trace the linkages between services during the development of both current system version and new versions for the system.

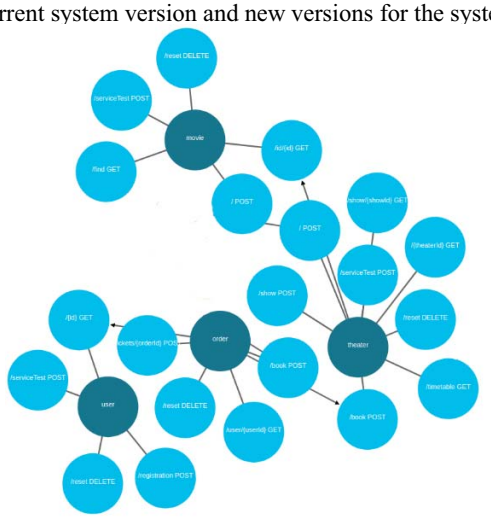

Figure 6 Visualized SDG using D3.js

C. Retrieval of SIC

As mentioned, in addition to SDG, we also designed a method to find the SIC (service invocation chains) to detect anomalies, divided into the following three steps:

1. Retrieve all service invocation paths: By querying the graph database (Neo4j), GMAT could find all service invocation paths containing one or more service calls.
2. Remove unnecessary paths: Some returned service invocation paths are not required to be kept since these paths are included in a larger path. As shown in Figure 7, the graph in the left-hand side shows interactions between endpoints in the database. Invocation paths in the right-hand side are the query

[1] https://neo4j.com/

[2] https://d3js.org/

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.

results. GMAT eliminates several paths (ones in red color) since these paths are merely sub-paths.

3. Group paths by the same starting service endpoint: We defined that an invocation chain is a graph comprising service calls for a leading service endpoint. As shown in Figure 7, each boxed group is defined as an SIC.

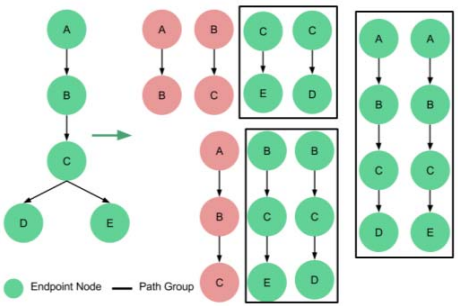

Figure 7 Concepts of SIC extraction

After performing the previous method of finding SIC, GMAT can detect the cyclic dependency problem using Tarjan's Strongly Connected Component algorithm [16]. Cycle dependency is potential design error, which may lead to resource dependency and competition and even cause unlimited service calls to make the system function improperly or even crash. Figure 8 shows an example of cyclic dependency. S_a, S_b, and S_c invoke one another to construct a cycle. GMAT will highlight this kind of problem in SDG.

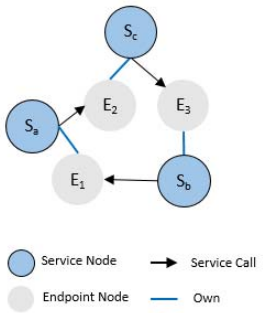

Figure 8 The concept of cyclic dependency

D. Calculation of Service Test Coverage

GMAT expects that developers test microservices based on Pact by using the *PactProviderRule* API and applying annotations of *PactVerification* and *Pact*. *PactProviderRule* is Pact's mock service which is used to verify the service call. *PactVerification* is executed by Pact and is expected to interact with the contract defined in Pact. The *Pact* annotation is an interactive contract that describes microservice capabilities expected. By using the *Reflection* mechanism, GMAT can inspect the test directory code of all microservice projects for a system, and determine whether all service calls have been tested using Pact in the above way. After examination, the failed service calls will be highlighted (in yellow) in SDG (See Figure 9).

After determining whether microservices satisfy the specified test cases, GMAT calculates the coverage of the service tests based on all the test results in the MSA-based system. Service test coverage is calculated by using formula 1.

$$msc(S_i) = \frac{tsc(S_i)}{sc(S_i)} \quad (1)$$

- S_i: the MSA-based system is being developed.
- $msc(S_i)$: service test coverage for the system S_i
- $tsc(S_i)$: the number of service calls that have been passed in the test for the system S_i.
- $sc(S_i)$: the number of all service calls for the system S_i.

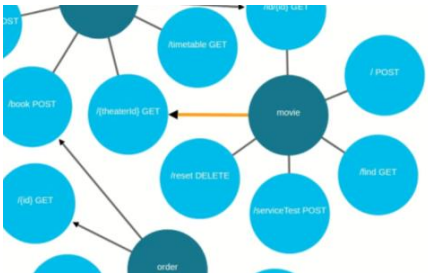

Figure 9 Highlighted failed service test in SDG

The developer could browse the SDG with test results to visually check the failed service calls. He/she could fix the bug by examining the service invocation chain and identifying the error root.

IV. Experimental Evaluations

The scale of a MSA system could range from less than ten microservices to hundreds of microservices (e.g., Netflix includes more than five hundreds microservices [17, 18]). Therefore, we need to test the performance of the GMAT system applied in various sized MSA-based systems. Currently, we cannot obtain the source code and configurations of real-world MSA-based systems, so that we decide to generate a set of test data (See Table 1) to perform simulation. The setup of the experiment was as follows:

1. Generate 11 simulated MSA-based applications with various number of microservices: 10, 20, 30, 40, 50, 100, 200, 300, 400, 500, and 1000.
2. Generate service endpoints randomly according to the distribution of numbers of service endpoints for the services published in APIs.guru[3], the most popular public service repository.
3. Allocate service calls between microservices randomly according to the distribution of numbers of service calls in the Netflix system.
4. Create visualized SDG for each project and record the time spent.
5. Hardware and software configurations:
   - OS: Arch Linux
   - CPU: Intel Core i7-4790
   - RAM: 16GB
   - Software Version: Java 1.8, Spring Boot 1.5.2, and Neo4j 3.1.1

[3] https://APIs.guru/openapi-directory/

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.

- Neo4j Driver: Bolt
- Neo4j Memory Pagecache Size: 4G
- Neo4j Memory Heap Max Size: 8G

Table 1. Test data for simulation

Service number	Endpoint number	Service call number (each service)	SDG creation time (second)
10	220	0	0.5
20	414	1	0.8
30	641	1	0.85
40	768	2	1.09
50	993	2	1.22
100	2,310	7	2.67
200	4,699	27	6.9
300	7,255	60	13.17
400	9,303	104	21.19
500	10,834	137	27.22
1,000	21,041	480	87.23

The experiment was conducted 5 times to obtain average results. Figure 10 presents the data related to the creation times of SDGs for the prepared MSA-based applications with various number of microservices. When the number of microservices is less than 200, the SDG creation time is less than seven seconds. Even when the number of microservices is one thousand, the creation time is less than 1.5 minutes. Our results indicate that GMAT yields acceptable performance for the creation of service dependency graph. It also demonstrate that GMAT is feasible to be applied in both small-scale and large-scale MSA-based applications.

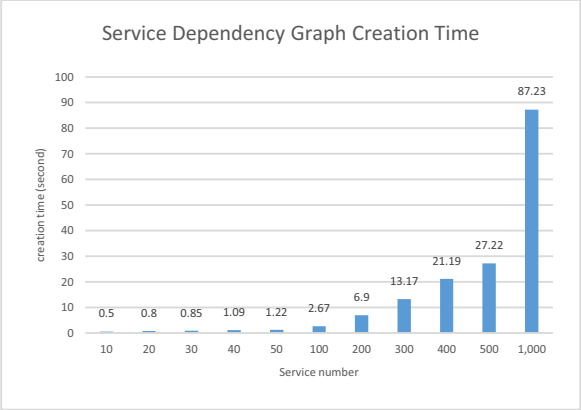

Figure 10 SDG creation time

V. Conclusions

In this paper, we propose an approach for assisting the development of MSA-based systems, referred to as GMAT (Graph-based Microservice Analysis and Testing). GMAT can automatically generate "Service Dependency Graph (SDG)" to analyze and visualize the dependency relationships between microservices. GMAT could help developers to manage the complex service interactions and effectively find error roots when failures occur. Experiments show that GMAT is able to work well for both small-scale and large MSA-based systems.

The next stage of development will add requriement scenarios to SDG for further improving the capability of change management for the target MSA-based system.

Acknowledgment

This research was sponsored by Ministry of Science and Technology in Taiwan under the grants MOST 105-2221-E-019-054-MY3 and MOST 106-2221-E-035 -014 -MY2.

References

[1] C. Y. Fan and S. P. Ma, "Migrating Monolithic Mobile Application to Microservice Architecture: An Experiment Report," in *2017 IEEE International Conference on AI & Mobile Services (AIMS)*, 2017, pp. 109-112.

[2] B. Familiar, "What Is a Microservice?," in *Microservices, IoT, and Azure*, ed: Springer, 2015, pp. 9-19.

[3] S. Newman, *Building Microservices*: " O'Reilly Media, Inc.", 2015.

[4] J. Lewis and M. Fowler, "Microservices," ed: Recuperado de: http://martinfowler.com/articles/microservices.html, 2014.

[5] C. Richardson, "Microservices: Decomposing applications for deployability and scalability," ed: InfoQ, 2014.

[6] J. Lewis and M. Fowler. (2014). *Microservices - a definition of this new architectural term*. Available: https://martinfowler.com/articles/microservices.html

[7] F. Montesi and J. Weber, "Circuit Breakers, Discovery, and API Gateways in Microservices," *arXiv preprint arXiv:1609.05830,* 2016.

[8] M. Soni, "End to End Automation on Cloud with Build Pipeline: The Case for DevOps in Insurance Industry, Continuous Integration, Continuous Testing, and Continuous Delivery," in *2015 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, 2015, pp. 85-89.

[9] M. Fowler and M. Foemmel, "Continuous integration," *Thought-Works) http://www. thoughtworks. com/Continuous Integration. pdf,* p. 122, 2006.

[10] H. Kang, M. Le, and S. Tao, "Container and Microservice Driven Design for Cloud Infrastructure DevOps," in *2016 IEEE International Conference on Cloud Engineering (IC2E)*, 2016, pp. 202-211.

[11] *What is Docker*. Available: https://www.docker.com/what-docker-old

[12] *OpenAPI Specification*. Available: https://github.com/OAI/OpenAPI-Specification/blob/master/versions/2.0.md

[13] H. Vural, M. Koyuncu, and S. Guney, "A Systematic Literature Review on Microservices," Cham, 2017, pp. 203-217.

[14] O. Shelajev. (2016). *Why Spring is Winning the Microservices Game*. Available: https://zeroturnaround.com/rebellabs/why-spring-is-winning-the-microservices-game/

[15] H. Schlosser. (2017). *Microservices trends 2017: Strategies, tools and frameworks*. Available: https://jaxenter.com/microservices-trends-2017-survey-133265.html

[16] R. Tarjan, "Depth-First Search and Linear Graph Algorithms," *SIAM Journal on Computing,* vol. 1, pp. 146-160, 1972.

[17] T. Mauro. (2015). *Adopting Microservices at Netflix: Lessons for Architectural Design*. Available: https://www.nginx.com/blog/microservices-at-netflix-architectural-best-practices/

[18] B. Wong. (2014). *The Case for Chaos*. Available: https://www.slideshare.net/BruceWong3/the-case-for-chaos

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:13:58 UTC from IEEE Xplore. Restrictions apply.